SOFTWARE QUALITY ASSURANCE

UI & API Test Automation Challenge

Framework Design • Test Engineering • Reliability • Parallel Execution

Assignment detail	Requirement
Assignment period	October 23 – November 2, 2026
Format	Team project, implementation, demonstration, and technical presentation
Required testing	UI automation and API automation
Tools	Playwright, Selenium, Robot Framework, Cypress, Appium – Android
Programming language	Team-selected; must be supported by the assigned tool

Submission deadline: November 2, 2026

1. Assignment Overview

Your team is required to design, implement, execute, and demonstrate an automated software testing solution for a real or publicly accessible application. The purpose is to apply Software Quality Assurance concepts in a practical project that resembles a real software testing environment.

You must do more than write automation scripts. You must design a maintainable framework, select appropriate test scenarios, validate application behavior, manage test data, execute tests independently and in parallel, investigate failures, and explain your technical decisions.

Each team has already been assigned an automation tool and a testing area. Use the assigned tool for the relevant UI or mobile automation work. Teams may select any suitable programming language supported by the tool.

Assigned automation tools

- Playwright: “Team Gray”
- Selenium: “Team Purple & Blue”
- Robot Framework: “Team Green”
- Cypress: “Team Yellow”
- Appium – Android: “Team Red”

Programming language: Your team is free to select a language supported by your assigned tool. Justify the selection based on technical suitability, maintainability, team experience, and ecosystem support.

2. Project Objectives

1. Select a suitable application and evaluate its testability.
2. Design an automation framework using appropriate software engineering principles.
3. Implement UI automation and API automation.
4. Create meaningful positive, negative, boundary, validation, and data-driven tests.
5. Apply appropriate assertions and verification techniques.
6. Design independent tests that can execute in parallel.
7. Apply (POM), SOLID principles, OOP, or equivalent architectural practices where appropriate.
8. Organize test code using consistent naming conventions and maintainable structures.
9. Investigate failures, identify root causes, and demonstrate corrective actions.
10. Explain and defend technical decisions during a live demonstration.

3. Application Selection Requirements

Your team may select any suitable application, including a demo application, a public web application, a mobile application, or an application developed by your team. The application must support the controls and behaviors required for your assigned testing area.

For the API component, the application must expose an accessible API, or you must use a related application and API that can be tested legitimately.

Mandatory UI elements

Element	Minimum requirement
Text input	Enter, edit, clear, and validate text.
Number input	Validate numeric values, boundaries, and invalid input where supported.
Text area	Enter and validate multi-line content.

File upload	Upload a file and verify the resulting behavior.
Image upload	Upload an image and validate acceptance or rejection.
Dropdown	Select options and verify the selected value.
Date picker	Select dates and validate restrictions or formats.
Dialog	Handle a modal, confirmation dialog, alert, or equivalent.
Dynamic HTML elements	Handle elements that appear, disappear, update, or change state dynamically.

Important: The application must provide actual functionality for these controls. Merely displaying a control is not sufficient. Tests must verify meaningful behavior, such as validation messages, uploaded file details, updated results, or changes in application state.

For mobile teams, select an Android application with appropriate native or hybrid controls. If an exact control is unavailable, obtain instructor approval for an equivalent feature that demonstrates the same testing concept.

Application selection checklist

- ☐ The application is accessible and stable enough for repeated testing.
- ☐ All mandatory UI elements are available, or approved equivalents have been agreed upon.
- ☐ The application provides sufficient scenarios for the required test counts.
- ☐ API endpoints are accessible and suitable for automated testing.
- ☐ The application permits testing without violating its terms of use.
- ☐ Test data can be created, isolated, and cleaned up where necessary.

Submit the application name, URL or installation details, selected language, and a short explanation of why the application is suitable.

4. UI Automation Requirements

Implement 8–10 successful UI test cases covering multiple testing types. In addition, implement one negative or failure-oriented test case and demonstrate how an issue is investigated and resolved.

4.1 Required UI test coverage

Test type	Example
Positive testing	Submit a form with valid information.
Negative testing	Submit a form with invalid or missing information.
Boundary testing	Enter values at, below, and above an allowed limit.
Field validation	Verify required fields, formats, and validation messages.
File validation	Upload a supported file and reject an unsupported file.
Date validation	Verify allowed and disallowed dates.
Dynamic element testing	Wait for a result, notification, or element to appear or update.
Dialog handling	Confirm or cancel an operation and verify its outcome.
State verification	Verify submitted information is displayed or persisted.
Regression testing	Verify an existing feature continues to work after a change.

The submitted suite must cover the mandatory UI elements across its test cases. A test may cover multiple related behaviors if each assertion is clear and meaningful.

4.2 Success test cases

Each test case must contain:

- A unique test ID and descriptive name.
- Preconditions, if applicable.

- Test data.
- Execution steps or a clear test procedure.
- Expected results.
- Actual results from execution.
- Assertions and validation evidence.

4.3 Failure-oriented test case

Examples include:

- Uploading an unsupported file.
- Entering an invalid number.
- Submitting an incomplete form.
- Selecting a prohibited date.
- Attempting to submit duplicate data.

The test must verify expected error handling, such as a validation message or rejection of the operation. You must also demonstrate one genuine automation failure, or an instructor-approved reproducible failure, and explain:

11. What failed and how the failure was detected.
12. Whether the cause was an application defect, test-data problem, synchronization issue, incorrect locator, environment issue, or faulty assertion.
13. How you investigated the root cause.
14. What corrective action you took.
15. How you verified the correction and prevented similar failures.

Do not simply change the expected result to make a failing test pass. The correction must be justified by intended application behavior.

4.4 Data-driven testing

At least one UI test case must use a data-driven approach. The same test logic must execute against multiple data sets without duplicating the entire test implementation.

Possible data sources include JSON, CSV, Excel, a structured array or test-data object, or a supported test framework data provider.

Your demonstration must explain the test data, expected results for each data set, and how the test reports each execution.

4.5 Assertions and validation

Assertions must verify application behavior, not merely confirm that a button can be clicked or a page can be opened.

- Expected page title or heading.
- Correct URL or navigation destination.
- Correct validation message.
- Correct selected dropdown option.
- Uploaded file name or upload status.
- Correct date or date restriction.
- Dialog visibility and expected action.
- Correct result after dynamic content changes.
- Correct data displayed after submission.
- Relevant API or application state where appropriate.

Use assertions supported by your assigned framework and explain why each important assertion is necessary.

5. API Automation Requirements

Implement 4–8 successful API test cases covering multiple testing types. Tests must validate API behavior and responses, not merely verify that the server is reachable.

5.1 Required API test coverage

Test type	Example
Status code	Verify the expected HTTP status code.
Response body	Validate required fields and values.
Schema or contract	Validate response structure and data types.
Data validation	Verify required fields, formats, and accepted values.
Boundary testing	Test minimum, maximum, or boundary values.
Negative testing	Verify appropriate responses to invalid requests.
Authentication or authorization	Verify access restrictions when supported.
CRUD operations	Validate create, read, update, and delete behavior when supported.
Error handling	Verify appropriate error responses and messages.
Headers	Validate relevant content type or other required headers.

Select cases that match the API's documented or intended behavior. Do not invent requirements the API does not support.

5.2 API test implementation

For every API test case, include:

- A unique test ID and descriptive name.
- The HTTP method and endpoint.
- Preconditions and required authentication, if applicable.
- Request headers, parameters, and body where relevant.
- Expected status code and response behavior.
- Assertions for the response body, schema, headers, or other relevant properties.
- Execution results and evidence.

5.3 API quality requirements

- Use a clear and reusable structure for requests and assertions.
- Separate environment configuration and test data from test logic where appropriate.
- Avoid hardcoding secrets or sensitive credentials in source code.
- Validate meaningful response properties rather than status codes alone.
- Use independent tests and controlled test data.
- Support parallel execution where the selected framework and environment allow it.
- Include appropriate negative testing, even though the required count of 4–8 refers to successful test cases.

Note: A successful negative test sends an invalid request and correctly verifies the expected error response.

6. Framework Architecture and Coding Standards

Develop an organized automation framework rather than a collection of unrelated scripts. Explain and demonstrate your architectural decisions.

6.1 Framework structure

Your framework should separate responsibilities wherever appropriate. Depending on the selected tool and language, it may include:

- Test cases and test suites.
- Page objects, screen objects, or equivalent UI abstractions.
- Reusable UI actions and utilities.
- API clients or request helpers.
- Test data and data providers.
- Environment configuration.
- Assertions and validation helpers.
- Setup, teardown, and cleanup logic.
- Test reports and execution evidence.

Illustrative project structure for JavaScript or TypeScript:

```
automation-project/
├── config/
├── tests/
│   ├── ui/
│   └── api/
├── pages/
├── screens/
├── clients/
├── fixtures/
├── test-data/
├── utils/
├── reports/
├── evidence/
├── .env.example
├── README.md
└── package.json
```

This is illustrative, not mandatory. Adapt it to your assigned tool and language. Do not create empty folders or unnecessary abstraction layers simply to match the example.

6.2 Design principles

- Page Object Model (POM): Separate UI interaction logic from test scenarios where suitable.
- Object-Oriented Programming (OOP): Use appropriate encapsulation and abstraction where beneficial.
- SOLID principles: Apply relevant principles to improve responsibility separation and maintainability.
- DRY: Avoid unnecessary duplication of logic.
- Single responsibility: Keep modules focused on clear purposes.
- Configuration management: Keep environment-specific values separate from test logic.
- Naming conventions: Use consistent, meaningful names for tests, variables, functions, classes, files, and fixtures.
- Error handling: Handle failures appropriately without concealing genuine defects.

You do not need to apply every design pattern everywhere. Explain why your chosen architecture is appropriate for the size and complexity of your project.

6.3 Locator and synchronization strategy

Explain how your framework identifies UI elements and handles asynchronous behavior, including:

- Locator selection and stability.

- Preference for accessible roles, labels, or stable test attributes where available.
- Avoidance of fragile selectors tied to page layout or generated IDs.
- Explicit or framework-supported waiting strategies.
- Handling dynamic elements and changing application state.
- Avoidance of unnecessary fixed sleeps.
- Approaches to debugging flaky tests.

For mobile projects, also explain the selection of appropriate Android locators and how the framework interacts with the device or emulator.

6.4 Code quality

Source code must be readable, consistently formatted, and organized. Include a README explaining how to install dependencies, configure the environment, execute tests, generate reports, and run the suite in parallel. The instructor must be able to run the project on a clean setup by following the documented instructions.

7. Test Independence and Parallel Execution

Your test suite must support independent execution and parallel mode. Tests must not depend on execution order and must be able to run concurrently without interfering with one another.

Required practices

- Each test establishes its own required preconditions.
- Tests do not depend on another test having run first.
- Use isolated test data or unique records where appropriate.
- Avoid shared mutable state between parallel tests.
- Avoid relying on a fixed record ID or data left behind by another test.
- Use appropriate setup and teardown procedures.
- Clean up created data when feasible and safe.
- Use the framework's supported parallel execution mechanism.
- Document genuine application or environment limitations.

Required demonstration

16. A normal test execution.
17. A parallel execution using the framework's supported configuration.
18. Evidence of multiple tests executing concurrently.
19. How the test design prevents data conflicts.
20. How you would investigate a test that fails only during parallel execution.

If the application cannot safely support full parallel execution, explain the limitation and demonstrate parallel execution using an approved isolated environment or test subset.

8. Reporting and Evidence

Provide clear evidence that the automation actually ran. Submit relevant execution results, such as:

- Test summary and pass/fail counts.
- Individual test results.
- Failure messages and stack traces where available.
- Screenshots or other evidence for important UI failures.

- API request and response evidence with secrets removed.
- Reports showing data-driven iterations.
- Evidence of parallel execution.

Do not expose passwords, tokens, private data, or other sensitive information in screenshots, reports, or source code. Reports must reflect actual execution results; do not manually change results to make the project appear successful.

9. Team Deliverables

A. Automation source code

Submit the complete UI and API automation source code to the email (muhammad.fatani@ext.junia.com) , including necessary configuration, dependencies, test data, and framework components. The instructor must be able to run the tests and inspect the implementation.

B. PowerPoint presentation

21. Project title and team members.
22. Selected application and reason for selection.
23. Assigned automation tool and programming language.
24. Why the team selected its programming language.
25. Framework architecture and project structure.
26. Design principles and naming conventions.
27. Test strategy and types of testing implemented.
28. UI automation test coverage and results.
29. API automation test coverage and results.
30. Data-driven testing implementation.
31. Independent tests and parallel execution.
32. Challenges encountered and how they were resolved.
33. One failure investigation, root cause, and corrective action.
34. Each member's individual contribution.
35. Lessons learned and possible improvements.

Every team member's contribution must be explicitly identified in the slides. General statements such as 'everyone helped with testing' are not sufficient.

C. Live demonstration

During the demonstration, your team must be able to:

- Launch the application or test environment.
- Execute UI automation tests.
- Execute API automation tests.
- Show assertions and validation logic in the code.
- Demonstrate the data-driven test.
- Demonstrate parallel execution.
- Explain the failure investigation and corrective action.
- Show results and reports.
- Answer technical questions about the implementation.

D. README and execution instructions

Provide clear instructions for setting up and executing the project, including prerequisites, configuration, commands, reporting, and parallel execution.

E. Test case documentation

Provide a test case inventory covering all submitted UI and API scenarios. Include test IDs, descriptions, test types, expected results, and execution status.

F. Individual contribution record

Document the area each member worked on, the files or modules they contributed to, and the work they can personally explain during the demonstration.

10. Team Member Responsibilities

Each team member must have a clearly defined responsibility. Possible areas include:

- UI framework architecture and reusable components.
- UI test implementation and validation.
- API client implementation and API test scenarios.
- Test data and data-driven testing.
- Parallel execution, configuration, and reporting.
- Failure analysis, documentation, and presentation.

These are examples only. Team members may share responsibilities, but each person must have identifiable contributions. Every member is expected to understand the overall framework, not just the part they implemented.

The instructor may ask any team member to explain or modify a portion of the code. Being assigned to presentation or documentation does not exempt a member from understanding the automation implementation.

Final Challenge

Your goal is not simply to produce a collection of scripts that pass once. Your goal is to demonstrate that your team can build an automation solution that another tester could understand, execute, maintain, and extend.

> ***"If the application changes tomorrow, how much of your automation framework will need to change, how will you detect the impact, and how confident are you that your tests will identify genuine defects without producing unnecessary failures?"***

Your answer should be supported by your framework design, code, execution results, and testing decisions.

Good luck, and build your automation as if another QA engineer will maintain it after you graduate.